Khulna University of Engineering & Technology

Department of Computer Science & Engineering

Course title: Compiler Design Laboratory

Course code: CSE 3212

Subject: Project idea submission

Submitted By:	**Submitted To:**
Name: Siyam Khan Roll: 2107120 Group: B2 Year: 3rd Term: 2nd Dept: CSE	**Mr. Md. Bodiuzzaman Shuvo** Lecturer, Department of Computer Science & Engineering, KUET **Ms. Subah Nawar** Lecturer, Department of Computer Science & Engineering, KUET

GenZ Programming Language Compiler

Introduction

The GenZ Programming Language Compiler is an innovative compiler project that bridges contemporary internet culture with formal programming education. This report documents the design, implementation, and analysis of a complete lexical analyzer for a novel programming language that uses Generation Z slang as its core syntax. The project successfully demonstrates how modern colloquial language can be systematically mapped to traditional programming constructs, making coding more accessible and engaging for digital natives.

GenZ Language represents a paradigm shift in programming language design by replacing traditional English-based keywords with contemporary internet slang. This approach aims to:

- Lower the barrier to entry for programming beginners
- Make code more relatable to younger generations
- Demonstrate the universality of programming concepts
- Bridge informal digital communication with formal computational thinking

Programming languages have traditionally used English keywords due to historical and practical reasons. However, as programming becomes a fundamental literacy skill, there's growing interest in making it more accessible across linguistic and cultural boundaries. GenZ Language explores this frontier by leveraging the global vernacular of internet culture.

Complete Keyword Mapping with C Equivalents

1. Core Programming Constructs

GenZ Keyword	C Equivalent	Detailed Meaning & Usage	Example
bet	int, float, char	Variable declaration. From "you bet" meaning certainty. Used for any mutable variable.	bet age = 25 → int age = 25
fr	const	Constant declaration. From "for real" meaning permanent/true. Used for immutable values.	fr PI = 3.14 → const float PI = 3.14
era	void func()	Function definition. From "era" meaning a period/time. Defines reusable code blocks.	era add() → void add()
vibeCheck	if	Conditional statement. From checking if something has good vibes. Tests conditions.	vibeCheck x > 0 → if (x > 0)
aint	else if	Alternative condition. From "ain't" meaning "is not". Tests additional conditions.	aint x == 0 → else if (x == 0)
meh	else	Default case. From "meh" meaning indifference/neutral. Executes when no conditions match.	meh { ... } → else { ... }
loopin	for	Iteration loop. From "looping" content on social media. Repeats code block.	loopin i=0#i<5#i++ → for(i=0;i<5;i++)
yeet	return	Return statement. From "yeet" meaning to throw/return something. Returns value from function.	yeet result → return result
spill_tea	printf()	Output function. From "spilling tea" meaning sharing gossip. Prints to console.	spill_tea("Hi") → printf("Hi")
OG	main()	Main function. From "Original Gangster" meaning the original/starting point. Program entry point.	OG main() → int main()
checkIt	scanf()	Input function. From "check it out" meaning to examine/input. Reads user input.	checkIt(&x) → scanf("%d", &x)

2. Data Types

GenZ Type	C Equivalent	Size/Bits	Range	Example
inty	int	32-bit	-2,147,483,648 to 2,147,483,647	inty age = 21
floaty	float	32-bit	1.2E-38 to 3.4E+38	floaty pi = 3.14159
stringy	char[]	Variable	Array of characters	stringy name = "GenZ"
booly	bool	8-bit	true/false	booly flag = noCap
listy	int[]	Variable	Homogeneous collection	listy nums = [1,2,3]
mappy	struct	Variable	Key-value pairs	mappy student

3.Boolean Values

GenZ Value	C Equivalent	Meaning	Binary
noCap	true	Truth value (no lie)	1
cap	false	False value (lie)	0

4. Control Flow & Structure

GenZ Keyword	C Equivalent	Purpose
ghost	break	Exit loop/switch
skip	continue	Skip to next iteration
squad	array	Collection of elements

GenZ Keyword	C Equivalent	Purpose
slidein	#include	Import libraries
clout	struct/class	Data structure
levelUp	extends/:	Inheritance
npc	NULL	Null pointer/value
test	try	Exception handling start
caughtIn4k	catch	Exception handler
slayy	N/A	Success/validation

5. Access Modifiers

GenZ Modifier	C Equivalent	Visibility
lowkey	static (private)	Limited scope
highkey	Default (public)	Full access

6. Modern Slang Additions

GenZ Keyword	C Equivalent	Purpose
sus	assert()	Assertion check
mainCharacter	this/self	Self-reference

GenZ Keyword	C Equivalent	Purpose
glowUp	++	Increment
fallOff	--	Decrement
ratio	%	Modulo operation
vibe	while	While loop
bussin	new	Memory allocation
mid	default	Switch default case
ate	throw	Exception throwing
rizz	char	Character type

The GenZ Programming Language initiative presents a novel and engaging approach to bridging contemporary internet culture with traditional programming constructs. By mapping popular GenZ slang to standard C programming elements, the language enhances memorability, accessibility, and learner engagement, particularly for younger programmers who are already familiar with these cultural expressions. The development of this language relies on a Linguistic Mapping Strategy, where each keyword was chosen based on four guiding principles:

1. **Semantic Proximity:** Each slang term's meaning closely aligns with the programming concept it represents. For example, yeet conveys the act of throwing, making it a natural fit for the return statement.

2. **Cultural Relevance:** The terms are derived from language currently trending in social media and internet culture, ensuring the language feels modern and relatable.
3. **Memorability:** Keywords are short, catchy, and easy to recall, which reduces cognitive load and enhances learning efficiency.
4. **Unambiguity:** Each slang term is distinct and avoids conflict with other programming elements, preventing syntactic confusion.

Overall, this innovative approach demonstrates how linguistic and cultural resonance can complement technical education. By translating traditional programming syntax into terms that resonate with GenZ, learners can more easily grasp programming concepts, retain knowledge, and enjoy the coding process. It exemplifies a creative fusion of linguistics, culture, and technology, paving the way for educational programming languages that are not only functional but also culturally engaging.